

Universidad Autónoma de Chihuahua

Facultad de Ingeniería

Carrera: LICO

Sharding en MongoDB con Docker (`sample_mflix`)

Base de Datos Avanzadas

Docente: José Saúl de Lira Montes

Integrantes:

Ezequiel Adolfo Herrera Hernández — *385717*

Diego Iván Ibarra Lozoya — *373781*

5 de noviembre de 2026

Índice

1. Introducción y Arquitectura **2**
1.1. Modelo de Datos y Shard Key 2

2. Configuración Paso a Paso y Evidencias **4**
2.1. Paso 1: Levantamiento de Contenedores 4
2.2. Paso 2: Inicialización de los Replica Sets 4
2.2.1. Config Server 4
2.2.2. Shard 1 5
2.2.3. Shard 2 5
2.2.4. Shard 3 6
2.3. Paso 3: Conexión a Mongos y Registro de los Shards 6
2.4. Paso 4: Carga de la Colección `movies` 7
2.5. Paso 5: Agregar el Campo `id_movie` 7
2.6. Paso 6: Shard Key Hashed 8
2.7. Paso 7: Balanceo de Carga entre Shards 9

3. Operaciones CRUD en la Base de Datos Distribuida **11**
3.1. Create (Inserción) 11
3.2. Read (Consulta) 13
3.3. Update (Actualización) 15
3.4. Delete (Eliminación) 16

4. Conclusiones **18**

1 Introducción y Arquitectura

En esta práctica armamos un cluster de MongoDB con *sharding* dentro de Docker y lo usamos para trabajar con la colección `movies` de la base `sample_mflix`. La idea es ver de cerca cómo una base de datos puede repartir sus documentos entre varios servidores y, aun así, comportarse como una sola para quien la consulta [1].

El cluster está formado por cinco contenedores, cada uno con una función distinta:

- **Config Server (`configsvr`):** guarda los metadatos del cluster, es decir, qué parte de los datos vive en cada shard (puerto 28017 / interno 27017).
- **Shard 1 (`shard1`):** primer fragmento de datos (puerto 28018 / interno 27018).
- **Shard 2 (`shard2`):** segundo fragmento de datos (puerto 28019 / interno 27019).
- **Shard 3 (`shard3`):** tercer fragmento de datos (puerto 28021 / interno 27021).
- **Router (`mongos`):** es el punto de entrada. Recibe las operaciones del cliente y las manda al shard que corresponde (puerto 28020 / interno 27020).

Cada nodo se configuró como un *replica set* de un solo miembro [2]. Lo hicimos así para no saturar una sola computadora con demasiados contenedores; la estructura es la correcta, pero con un único miembro por conjunto no se puede ver la elección de un nuevo primario.

Esta fue la primera versión del cluster y es la que se documenta en los pasos de esta sección (carpeta `Docker-Compose/` del repositorio). Después la ampliamos a replica sets de tres nodos cada uno, para que sí hubiera un primario y dos secundarios (carpeta `cluster/` del repositorio, 13 contenedores en total y el `mongos` en el puerto 27117). Las operaciones CRUD de la sección 3 las hicimos sobre esa versión final.

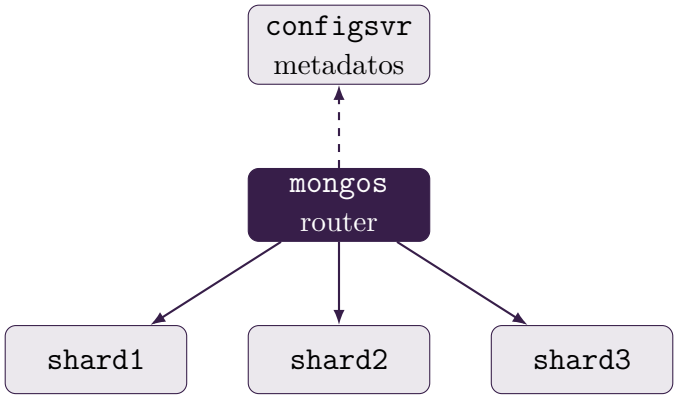


Figura 1: Arquitectura del cluster: el cliente solo habla con `mongos`, que consulta al `configsvr` y reparte el trabajo entre los tres shards.

1.1 Modelo de Datos y Shard Key

Trabajamos con la colección `movies` de `sample_mflix`, que tiene 21,349 películas. Como los documentos no traen un campo pensado para repartirlos, les agregamos nosotros uno nuevo, `id_movie`, con un número consecutivo del 1 al 21,349. Sobre ese campo definimos el *shard key* de tipo `hashed`:

```
{ id_movie: "hashed" }
```

Listing 1: Definición del shard key hashed

Con un shard key hashed, MongoDB no usa el valor tal cual, sino el resultado de aplicarle una función hash. Así, aunque los números sean consecutivos, los documentos se dispersan de forma pareja entre los shards en lugar de irse todos al mismo [3].

2 Configuración Paso a Paso y Evidencias

Estas son las operaciones que fuimos ejecutando, en orden, para dejar el cluster funcionando.

2.1 Paso 1: Levantamiento de Contenedores

Primero escribimos el `docker-compose.yml` con los cinco servicios, cada uno con su volumen para que los datos no se pierdan al apagar los contenedores [4]. Después los levantamos todos de golpe.

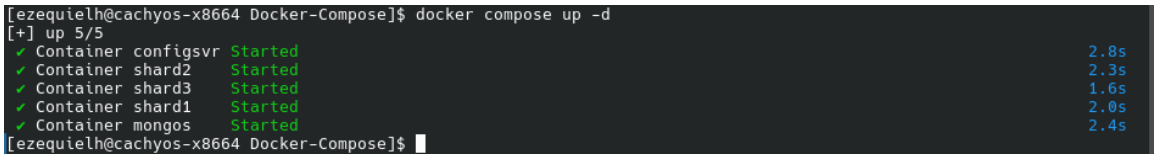


Figura 2: Inicio de los cinco contenedores con `docker compose up -d`.

2.2 Paso 2: Inicialización de los Replica Sets

Aunque los contenedores ya estén arriba, ningún nodo sabe todavía que es parte de un replica set. Por eso entramos a cada uno con `mongosh` y corrimos `rs.initiate()`. El `_id` tiene que coincidir con el nombre que se puso en `--replSet` en el compose, y el `host` lleva el nombre del contenedor con su puerto interno, porque los nodos se hablan entre ellos por la red de Docker.

2.2.1. Config Server

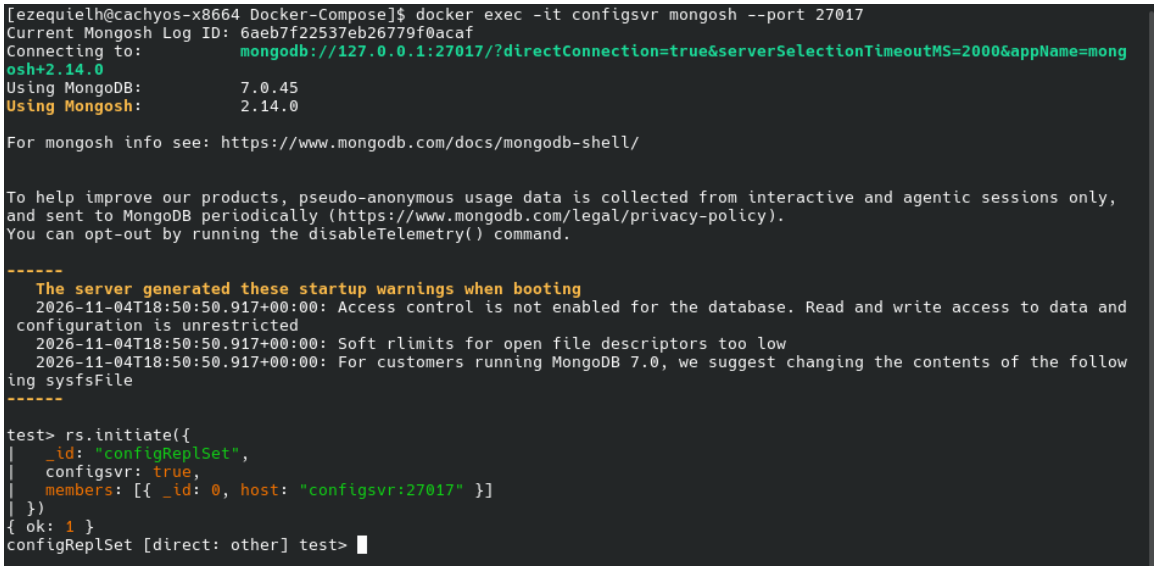


Figura 3: Inicialización del replica set del Config Server (`configReplSet`). Es el único que lleva `configsvr: true`.

2.2.2. Shard 1

```
[ezequielh@cachyos-x8664 Docker-Compose]$ docker exec -it shard1 mongosh --port 27018
Current Mongosh Log ID: 6aeb7fb703d0a8d3a275fe70
Connecting to:          mongodb://127.0.0.1:27018/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.14.0
Using MongoDB:          7.0.45
Using Mongosh:          2.14.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, pseudo-anonymous usage data is collected from interactive and agentic sessions only,
and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-11-04T18:50:50.299+00:00: Access control is not enabled for the database. Read and write access to data and
 configuration is unrestricted
   2026-11-04T18:50:50.299+00:00: Soft rlimits for open file descriptors too low
   2026-11-04T18:50:50.300+00:00: For customers running MongoDB 7.0, we suggest changing the contents of the follow
ing sysfsFile
------

test> rs.initiate({
|   _id: "shard1ReplSet",
|   members: [{ _id: 0, host: "shard1:27018" }]
| })
{ ok: 1 }
shard1ReplSet [direct: other] test>
```

Figura 4: Inicialización del replica set del Shard 1 (`shard1ReplSet`).

2.2.3. Shard 2

```
[ezequielh@cachyos-x8664 Docker-Compose]$ docker exec -it shard2 mongosh --port 27019
Current Mongosh Log ID: 6aeb801fb3d8d0d062f6eff2
Connecting to:          mongodb://127.0.0.1:27019/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.14.0
Using MongoDB:          7.0.45
Using Mongosh:          2.14.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, pseudo-anonymous usage data is collected from interactive and agentic sessions only,
and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-11-04T18:50:50.444+00:00: Access control is not enabled for the database. Read and write access to data and
 configuration is unrestricted
   2026-11-04T18:50:50.444+00:00: Soft rlimits for open file descriptors too low
   2026-11-04T18:50:50.445+00:00: For customers running MongoDB 7.0, we suggest changing the contents of the follow
ing sysfsFile
------

test> rs.initiate({
|   _id: "shard2ReplSet",
|   members: [{ _id: 0, host: "shard2:27019" }]
| })
{ ok: 1 }
```

Figura 5: Inicialización del replica set del Shard 2 (`shard2ReplSet`).

2.2.4. Shard 3

```
[ezequielh@cachyos-x8664 Docker-Compose]$ docker exec -it shard3 mongosh --port 27021
Current Mongosh Log ID: 6aeb80928d262865fa67480b
Connecting to:          mongodb://127.0.0.1:27021/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.14.0
Using MongoDB:          7.0.45
Using Mongosh:          2.14.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, pseudo-anonymous usage data is collected from interactive and agentic sessions only,
and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-11-04T18:50:50.299+00:00: Access control is not enabled for the database. Read and write access to data and
 configuration is unrestricted
   2026-11-04T18:50:50.299+00:00: Soft rlimits for open file descriptors too low
   2026-11-04T18:50:50.299+00:00: For customers running MongoDB 7.0, we suggest changing the contents of the follow
ing sysfsFile
------

test> rs.initiate({
|   _id: "shard3ReplSet",
|   members: [{ _id: 0, host: "shard3:27021" }]
| })
{ ok: 1 }
shard3ReplSet [direct: other] test>
```

Figura 6: Inicialización del replica set del Shard 3 (`shard3ReplSet`).

2.3 Paso 3: Conexión a Mongos y Registro de los Shards

Con los cuatro replica sets listos, reiniciamos `mongos` para que se conectara al config server y entramos a su consola. Desde ahí registramos los tres shards con `sh.addShard()`. A partir de este punto, todo lo que hagamos con el cluster se hace a través del router.

```
[ezequielh@cachyos-x8664 Docker-Compose]$ docker exec -it mongos mongosh --port 27020
Current Mongosh Log ID: 6aeb80f4b4f6807fd3cb519c
Connecting to:          mongodb://127.0.0.1:27020/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.14.0
Using MongoDB:          7.0.45
Using Mongosh:          2.14.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, pseudo-anonymous usage data is collected from interactive and agentic sessions only,
and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-11-04T19:00:53.076+00:00: Access control is not enabled for the database. Read and write access to data and
 configuration is unrestricted
------

[direct: mongos] test> sh.addShard("shard1ReplSet/shard1:27018")
| sh.addShard("shard2ReplSet/shard2:27019")
| sh.addShard("shard3ReplSet/shard3:27021")
{
  shardAdded: 'shard3ReplSet',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793818920, i: 11 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793818920, i: 11 })
}
```

Figura 7: Conexión a `mongos` y registro de `shard1ReplSet`, `shard2ReplSet` y `shard3ReplSet`.

2.4 Paso 4: Carga de la Colección `movies`

El archivo con las películas venía exportado en JSON, así que lo copiamos al contenedor de `mongos` y lo importamos con `mongoimport`. Como el archivo es un arreglo (`[ {...}, {...} ]`), hubo que agregar la opción `--jsonArray`.

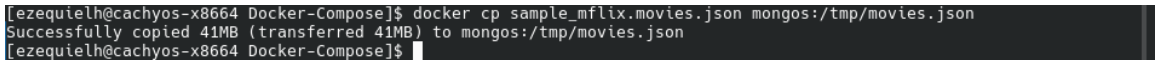


Figura 8: Copia del archivo JSON al contenedor `mongos` con `docker cp`.

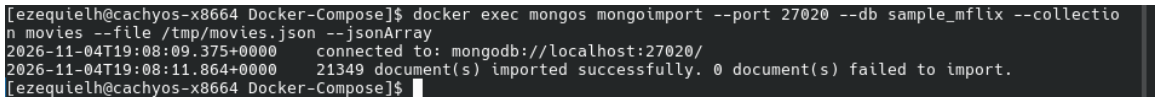


Figura 9: Importación de la colección: 21,349 documentos importados y ninguno fallido.

2.5 Paso 5: Agregar el Campo `id_movie`

Antes de fragmentar la colección había que asegurarse de que todos los documentos tuvieran el campo que va a ser el shard key. Recorrimos la colección con un cursor y a cada película le fuimos asignando un número consecutivo.

```
use sample_mflix

let contador = 1
db.movies.find().forEach(function(doc) {
  db.movies.updateOne(
    { _id: doc._id },
    { $set: { id_movie: contador } }
  )
  contador++
})
```

Listing 2: Asignación de `id_movie` a cada documento

Para comprobar que funcionó, contamos los documentos que no tuvieran el campo (el resultado fue 0) y revisamos una película cualquiera.



Figura 10: Asignación de `id_movie` y verificación: 0 documentos sin el campo.

2.6 Paso 6: Shard Key Hashed

Ahora sí activamos el sharding. Primero creamos el índice hashed sobre `id_movie`; esto va antes del `shardCollection` porque la colección ya tenía datos y MongoDB necesita el índice listo para poder usarlo como llave. Después habilitamos el sharding en la base de datos y, por último, en la colección.

```
db.movies.createIndex({ id_movie: "hashed" })
sh.enableSharding("sample_mflix")
sh.shardCollection("sample_mflix.movies", { id_movie: "hashed" })
```

Listing 3: Activación de sharding con shard key hashed



Figura 11: Creación del índice hashed y fragmentación de `sample_mflix.movies`.

2.7 Paso 7: Balanceo de Carga entre Shards

Al fragmentar la colección, todos los documentos quedaron juntos en un solo chunk dentro de `shard3`, que es el shard primario de la base de datos. Esto es normal: MongoDB parte los chunks hasta que llegan a 128 MB por defecto, y nuestra colección pesa unos 33 MB, así que el balanceador no veía nada que mover [5].

Para poder mostrar el balanceo, bajamos el tamaño máximo de chunk de esta colección a 1 MB, partimos el chunk en varios con `splitFind` y dejamos que el balanceador hiciera su trabajo.

```
db.adminCommand({
  configureCollectionBalancing: "sample_mflix.movies",
  chunkSize: 1
})

for (let i = 1; i <= 20000; i += 2000) {
  sh.splitFind("sample_mflix.movies", { id_movie: i })
}

sh.getBalancerState()
```

Listing 4: Preparación para forzar el balanceo entre shards

Antes del balanceo. Justo después del `shardCollection`, `shard3` tenía el 100 % de los datos y un solo chunk.

```
[direct: mongos] sample_mflix> db.movies.getShardDistribution()
Shard shard3ReplSet at shard3ReplSet/shard3:27021
{
  data: '32.79MiB',
  docs: 21349,
  chunks: 1,
  'estimated data per chunk': '32.79MiB',
  'estimated docs per chunk': 21349
}
---
Totals
{
  data: '32.79MiB',
  docs: 21349,
  chunks: 1,
  'Shard shard3ReplSet': [
    '100 % data',
    '100 % docs in cluster',
    '1KiB avg obj size on shard'
  ]
}
```

Figura 12: Distribución inicial: todos los documentos en `shard3`.

Durante el balanceo. Con los chunks ya partidos, el balanceador empezó a migrarlos hacia `shard1` y `shard2`. En la captura el reparto de datos ya es parejo (alrededor de 32 %, 30 % y 38 %), con 27 chunks en total.

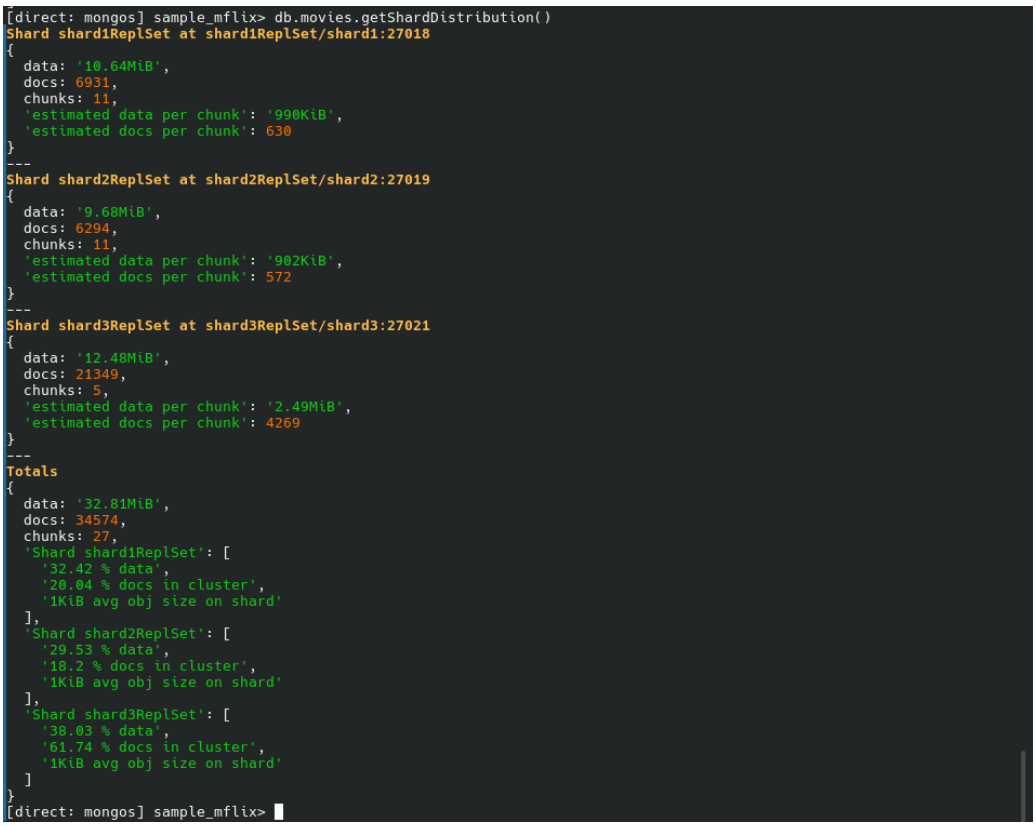


Figura 13: Distribución de la colección entre los tres shards mientras el balanceador migraba chunks.

El total de documentos que aparece en la captura (34,574) es mayor a los 21,349 reales. Esto se debe a que, cuando un chunk se mueve de shard, el shard de origen conserva una copia (documentos huérfanos) y la borra en segundo plano, por defecto 15 minutos después. Mientras tanto, `getShardDistribution()` cuenta esas copias. El conteo real, que pasa por `mongos`, sigue siendo de 21,349.

Después del balanceo. Cuando se limpiaron los documentos huérfanos, el total volvió a coincidir con la colección.

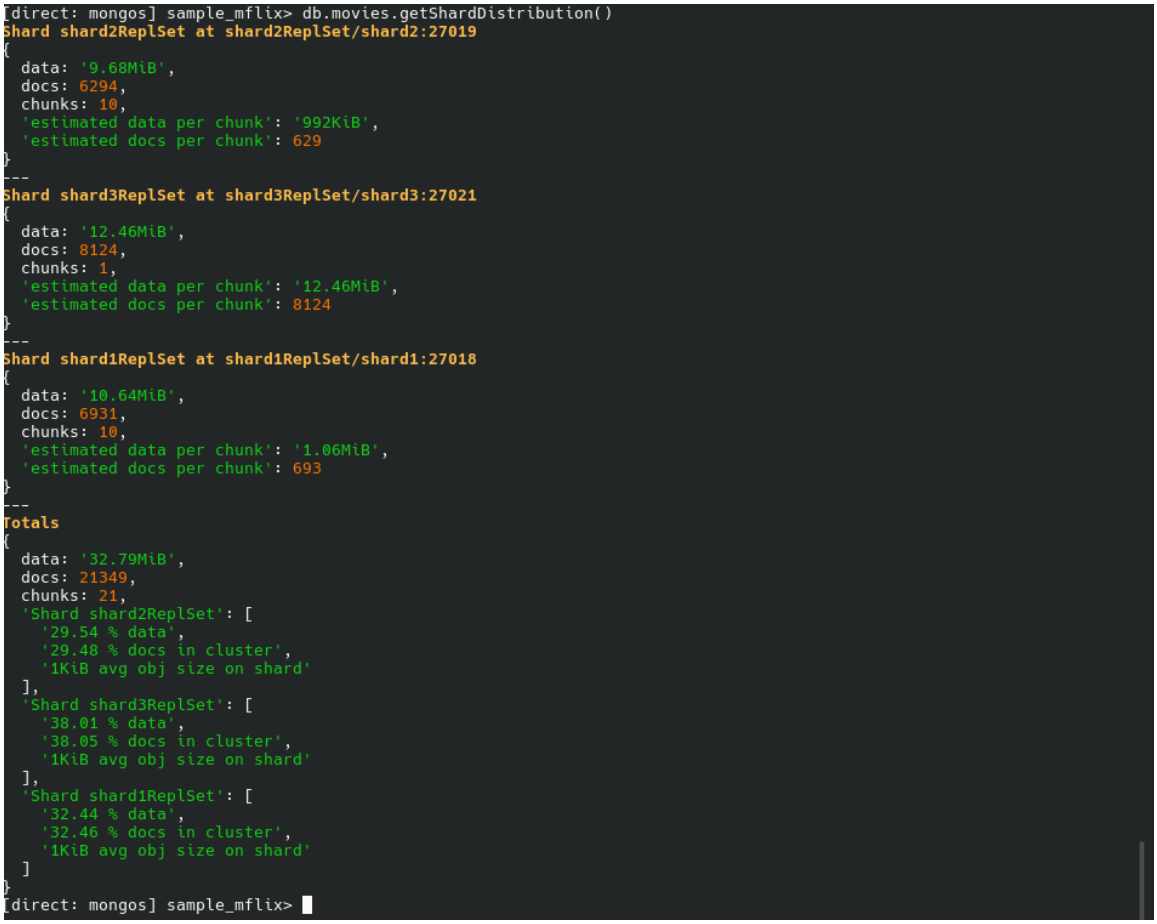


Figura 14: Distribución final: documentos repartidos entre los tres shards.

3 Operaciones CRUD en la Base de Datos Distribuida

En esta sección se presentan las operaciones de inserción, consulta, actualización y eliminación ejecutadas a través de `mongos`. Después de cada una revisamos con `db.movies.getShardDistribut` cómo quedaron los documentos entre los shards. Estas operaciones se hicieron sobre la versión final del cluster (replica sets de tres nodos, carpeta `cluster/` del repositorio), por eso en las capturas cada shard aparece con tres nodos.

3.1 Create (Inserción)

Primero insertamos una película con `insertOne` y luego otras diez con `insertMany`. El último `id_movie` de la colección era el 21,349, así que las nuevas llevan del 21,350 al 21,360 y el total pasó de 21,349 a 21,360 documentos. Para saber en qué shard cayó cada una recorrimos las películas de prueba y hicimos un `explain()` del `find` por `id_movie` de cada una: nueve quedaron en el shard3 (la 21,350, de la 21,352 a la 21,358 y la 21,360) y dos en el shard1 (la 21,351 y la 21,359). En el shard2 no cayó ninguna y se quedó con sus 8,369 documentos; con solo once documentos no se espera un reparto parejo.

```
// insertOne: una pelicula nueva (el ultimo id_movie era 21349)
db.movies.insertOne({
  id_movie: 21350, title: "El Ultimo Commit", year: 2026,
  genres: ["Drama", "Comedy"], runtime: 98, countries: ["Mexico"],
  imdb: { rating: 7.8, votes: 120 }, prueba: true
```

```
})

// insertMany: diez peliculas mas (21351 a 21360)
db.movies.insertMany([
  { id_movie: 21351, title: "Shard de Medianoche", year: 2026,
    genres: ["Thriller"], imdb: { rating: 6.9 }, prueba: true },
  { id_movie: 21352, title: "Replica en Chihuahua", year: 2026,
    genres: ["Drama"], imdb: { rating: 8.1 }, prueba: true },
  { id_movie: 21353, title: "El Router Perdido", year: 2026,
    genres: ["Adventure"], imdb: { rating: 5.2 }, prueba: true },
  { id_movie: 21354, title: "Balanceo de Carga", year: 2026,
    genres: ["Action"], imdb: { rating: 4.7 }, prueba: true },
  { id_movie: 21355, title: "Config Server", year: 2026,
    genres: ["Sci-Fi"], imdb: { rating: 7.3 }, prueba: true },
  { id_movie: 21356, title: "Hashed", year: 2026,
    genres: ["Horror"], imdb: { rating: 3.9 }, prueba: true },
  { id_movie: 21357, title: "Chunk 404", year: 2026,
    genres: ["Comedy"], imdb: { rating: 6.1 }, prueba: true },
  { id_movie: 21358, title: "La Llave del Shard", year: 2026,
    genres: ["Mystery"], imdb: { rating: 8.5 }, prueba: true },
  { id_movie: 21359, title: "Mongos y Compania", year: 2026,
    genres: ["Comedy", "Family"], imdb: { rating: 4.4 }, prueba: true },
  { id_movie: 21360, title: "Primary y Secondary", year: 2026,
    genres: ["Romance"], imdb: { rating: 7.0 }, prueba: true }
])

// en que shard quedo cada pelicula de prueba (explain del find por
   id_movie)
db.movies.find({ prueba: true }, { _id: 0, id_movie: 1 }).sort({
   id_movie: 1 }).forEach(p => print(p.id_movie + " -> " + db.movies.
   find({ id_movie: p.id_movie }).explain().queryPlanner.winningPlan.
   shards[0].shardName))
```

Listing 5: Comando de inserción



Figura 15: Inserción de las películas y shard donde quedó cada una.

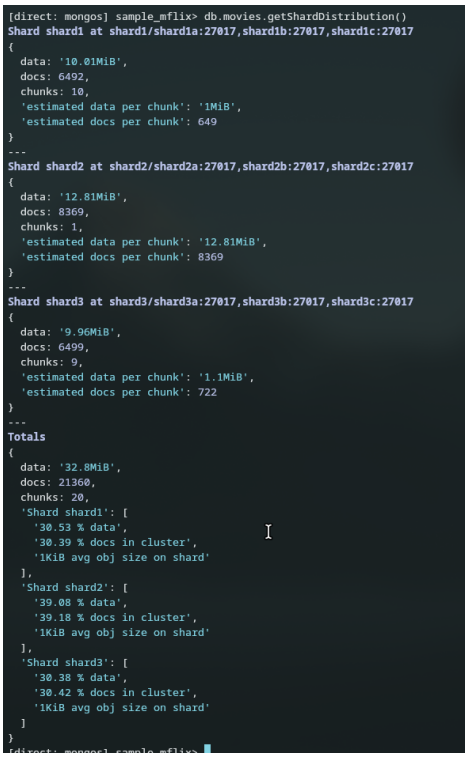


Figura 16: Distribución entre shards después de la inserción.

3.2 Read (Consulta)

Hicimos tres consultas y un aggregate. La búsqueda por `id_movie: 100` usa la shard key, así que el `mongos` sabe a qué shard ir y el plan sale como `SINGLE_SHARD` (shard1); regresó *Payment Deferred* (1932). La búsqueda por título de *The Godfather* (id 2223) no usa la shard key, entonces el `mongos` tiene que preguntarle a los tres shards y el plan

es SHARD_MERGE. Después consultamos las películas de 1994, que son 305, ordenadas por rating (en los primeros lugares salen dos registros de *The Shawshank Redemption* con 9.3, luego *Baseball* y *Pulp Fiction*). Por último hicimos un aggregate con los cinco géneros que tienen más películas, encabezado por Drama con 12,387. El total se mantuvo en 21,360 documentos y la distribución entre shards no cambió (6,492, 8,369 y 6,499).

```
// por shard key: el mongos manda la consulta a un solo shard
db.movies.find({ id_movie: 100 }, { _id: 0, id_movie: 1, title: 1, year: 1, genres: 1 })

// por titulo: no es la shard key, el mongos pregunta a todos los shards
db.movies.find({ title: "The Godfather" }, { _id: 0, id_movie: 1, title: 1, year: 1, "imdb.rating": 1 })

// por year: las 5 con mejor rating
db.movies.find({ year: 1994, "imdb.rating": { $type: "number" } }, { _id: 0, id_movie: 1, title: 1, "imdb.rating": 1 }).sort({ "imdb.rating": -1, id_movie: 1 }).limit(5)

// aggregate: los 5 generos con mas peliculas
db.movies.aggregate([
  { $unwind: "$genres" },
  { $group: { _id: "$genres", peliculas: { $sum: 1 }, rating_prom: { $avg: "$imdb.rating" } } },
  { $sort: { peliculas: -1 } },
  { $limit: 5 }
])
```

Listing 6: Comandos de consulta

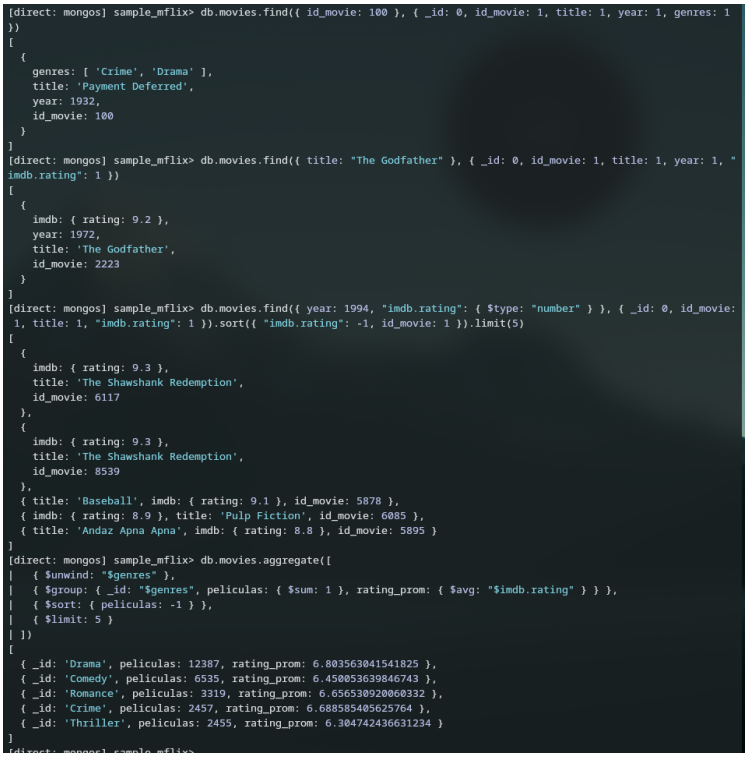


Figura 17: Evidencia de la operación Read.

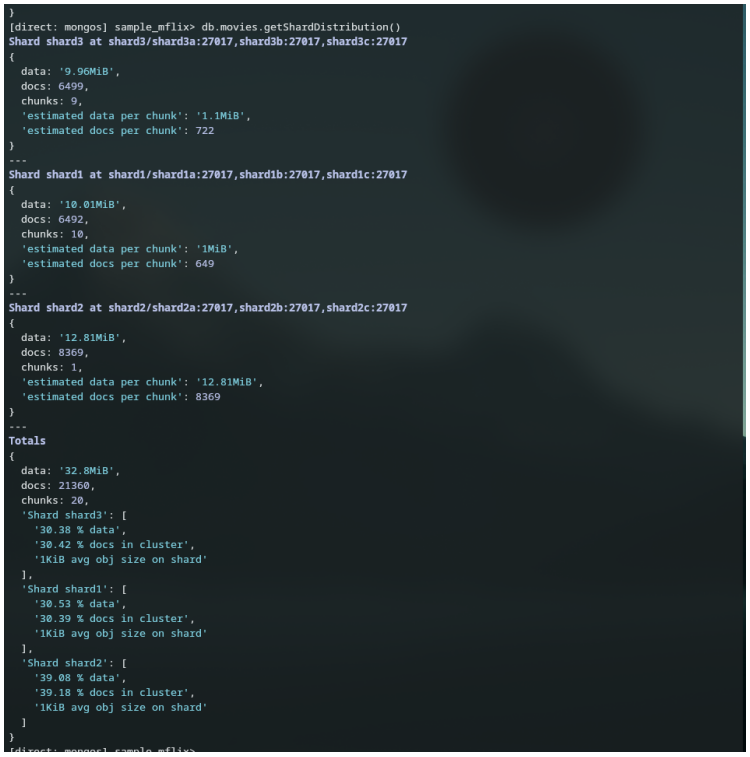


Figura 18: Distribución entre shards después de la consulta.

3.3 Update (Actualización)

Con `updateOne` cambiamos el rating (de 7.8 a 9.1) y el runtime (de 98 a 105) de la película 21,350. Después usamos `updateMany` para agregar el campo `estado` con valor *estreno* a las 11 películas de prueba que insertamos, y otro `updateMany` sobre datos originales para agregar el campo `epoca` con valor *cine mudo* a las 18 películas anteriores a 1920. Como estos filtros no llevan la shard key, el `mongos` los manda a todos los shards. El total siguió en 21,360 documentos y la distribución no cambió, porque actualizar estos campos no mueve documentos de un shard a otro.

```
// updateOne por shard key: cambia rating y runtime de la 21350
db.movies.updateOne({ id_movie: 21350 }, { $set: { "imdb.rating": 9.1,
    runtime: 105 } })

// updateMany: marca las peliculas de prueba (11)
db.movies.updateMany({ prueba: true }, { $set: { estado: "estreno" } })

// updateMany sobre datos originales: las anteriores a 1920 (18)
db.movies.updateMany({ year: { $lt: 1920 } }, { $set: { epoca: "cine
    mudo" } })
```

Listing 7: Comando de actualización

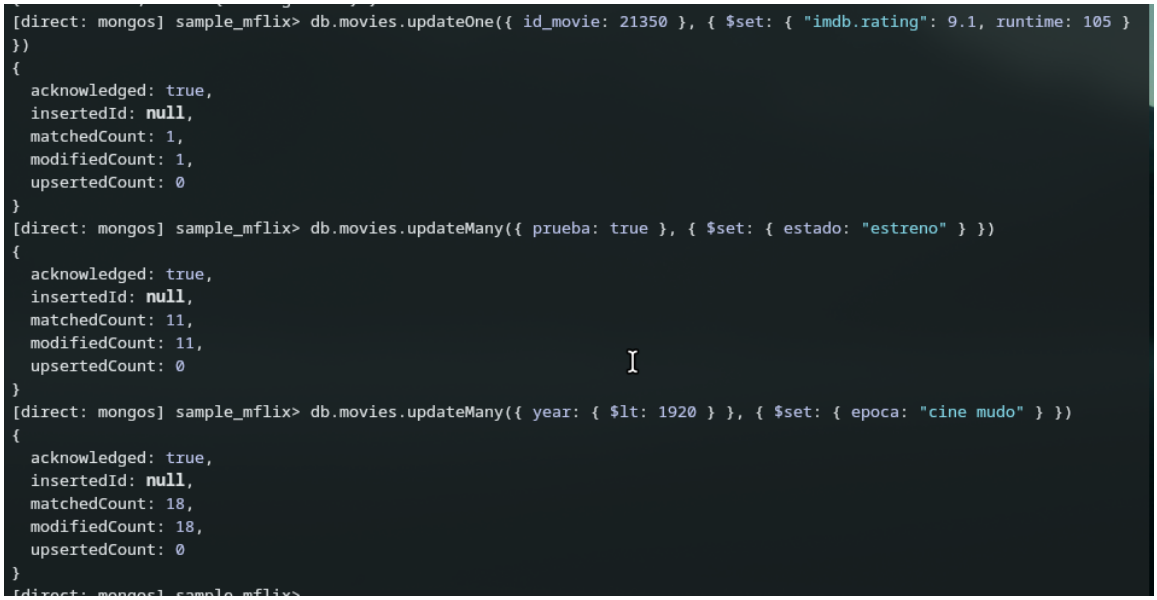


Figura 19: Evidencia de la operación Update.

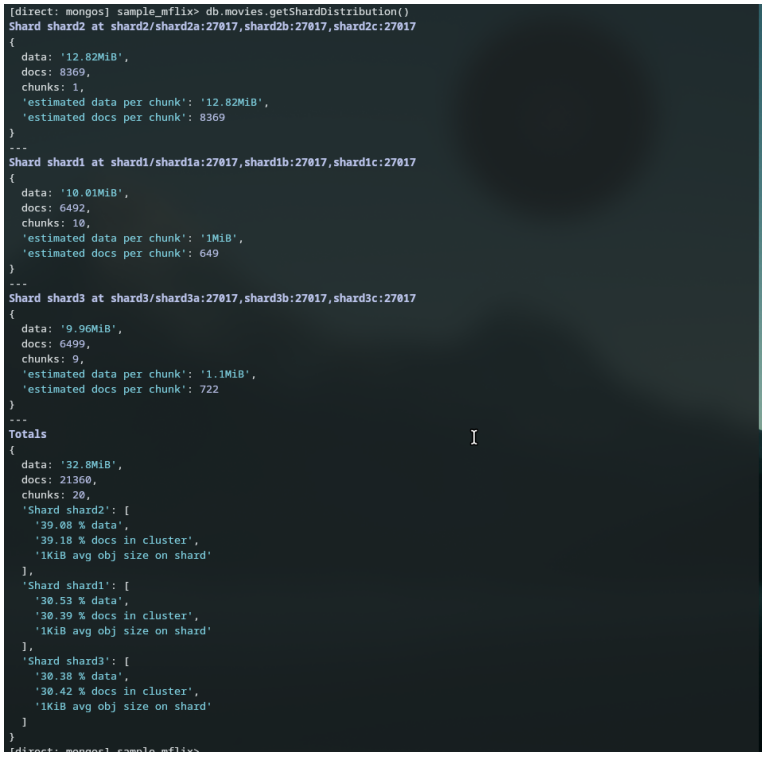


Figura 20: Distribución entre shards después de la actualización.

3.4 Delete (Eliminación)

Con `deleteOne` borramos *El Router Perdido*. Lo hicimos por `id_movie` (21,353) y no por título, porque en MongoDB 7.0 un `deleteOne` sobre una colección fragmentada necesita la shard key o el `_id` en el filtro, y si no marca error. Después, con `deleteMany`, eliminamos las películas de prueba con rating menor a 5: *Balanceo de Carga*, *Hashed* y *Mongos y Compania* (ids 21,354, 21,356 y 21,359). El total bajó de 21,360 a 21,356 documentos. Por shard, el shard1 pasó de 6,492 a 6,491, el shard2 se mantuvo en 8,369 y

el shard3 bajó de 6,499 a 6,496.

```
// deleteOne por shard key (en mongo 7.0 el filtro necesita id_movie o _id)
db.movies.deleteOne({ id_movie: 21353 })

// deleteMany: las de prueba con rating menor a 5 (3 peliculas)
db.movies.deleteMany({ prueba: true, "imdb.rating": { $lt: 5 } })
```

Listing 8: Comando de eliminación

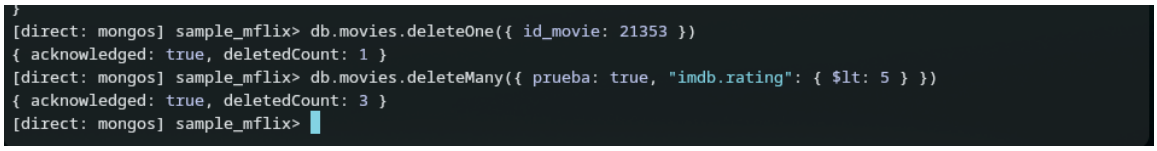


Figura 21: Evidencia de la operación Delete.

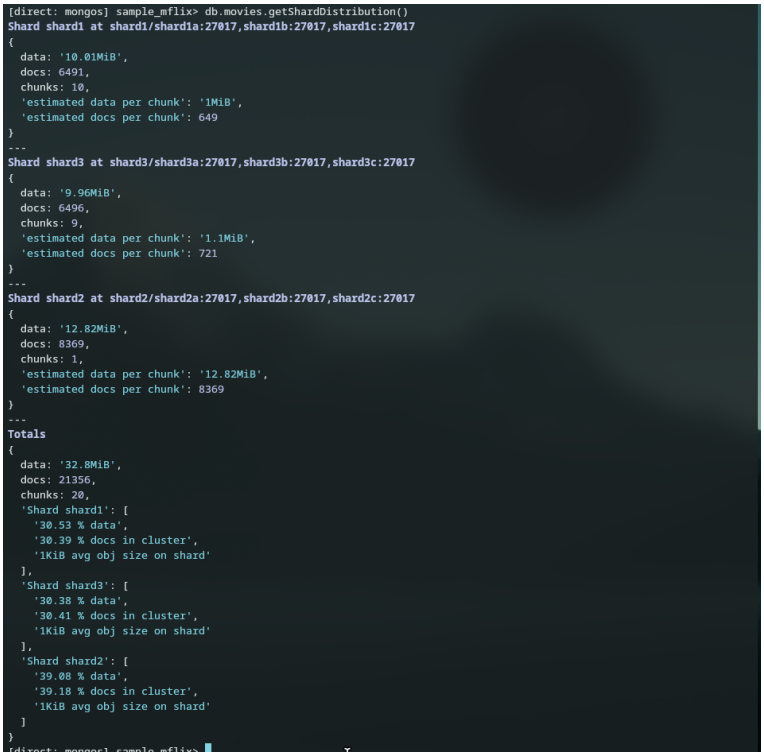


Figura 22: Distribución entre shards después de la eliminación.

4 Conclusiones

Con esta práctica pudimos ver, paso por paso, qué hace cada pieza de un cluster fragmentado: el config server guarda el mapa de dónde está cada dato, los shards almacenan los documentos y el router `mongos` esconde todo eso para que se trabaje como con una sola base de datos.

Lo que más nos llamó la atención fue el balanceo. Tras fragmentar la colección, todo se quedó en un solo shard, y el reparto parejo solo apareció cuando bajamos el tamaño de chunk y dejamos que el balanceador moviera los datos. También aprendimos que `getShardDistribution()` puede mostrar más documentos de los reales mientras se limpian los huérfanos, y que conviene confirmar el total con un conteo que pase por `mongos`.

En el CRUD vimos que las operaciones que llevan la shard key, como buscar o borrar por `id_movie`, el `mongos` las manda a un solo shard, mientras que las que filtran por otros campos (título, año) se reparten entre los tres. Las películas nuevas cayeron en el shard1 y el shard3, ninguna en el shard2, y al final la colección quedó con 21,356 documentos: los 21,349 originales más siete de prueba.

Referencias

[1] MongoDB, Inc., "Sharding — mongodb manual." https://www.mongodb.com/docs/manual/sharding/. Consultado: 4 de noviembre de 2026.

[2] MongoDB, Inc., "Replication — mongodb manual." https://www.mongodb.com/docs/manual/replication/. Consultado: 4 de noviembre de 2026.

[3] MongoDB, Inc., "Hashed sharding — mongodb manual." https://www.mongodb.com/docs/manual/core/hashed-sharding/. Consultado: 4 de noviembre de 2026.

[4] Docker, Inc., "Docker compose overview." https://docs.docker.com/compose/. Consultado: 4 de noviembre de 2026.

[5] MongoDB, Inc., "Sharded cluster balancer — mongodb manual." https://www.mongodb.com/docs/manual/core/sharding-balancer-administration/. Consultado: 4 de noviembre de 2026.